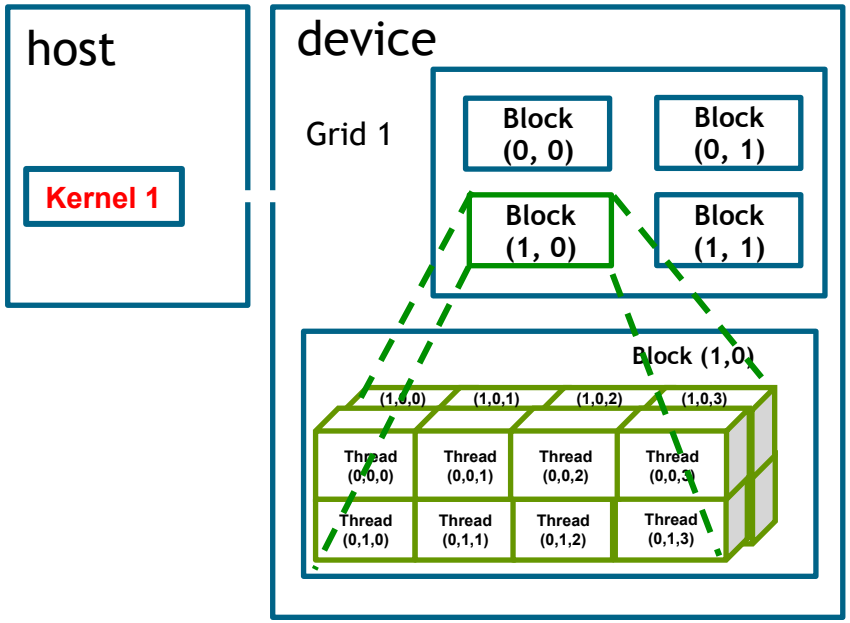
BLG562E: Parallel Computing For GPUs using CUDA

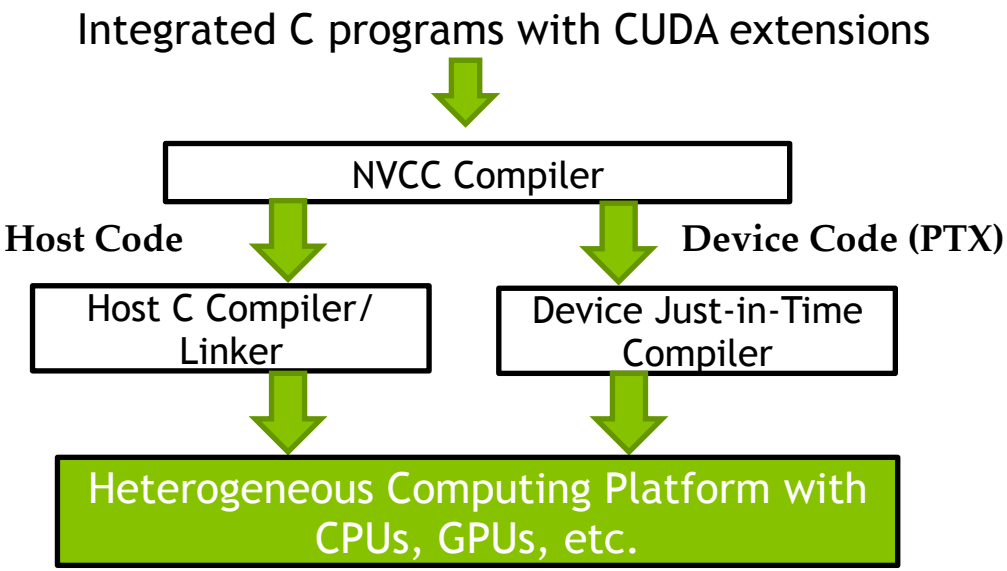
Spring 2026

Week 5- GPU Organization and CUDA Occupancy

CPU+ GPU Heterogeneous Computing



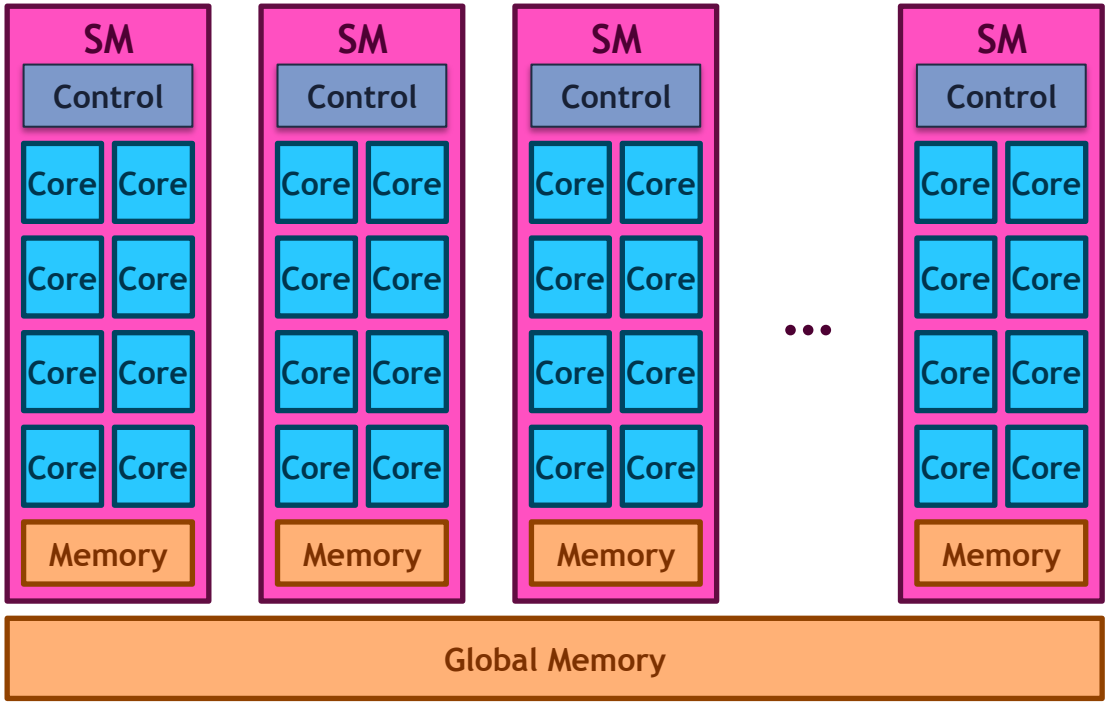
Compiling A CUDA Program



GPU Architecture

A GPU consists of multiple **Streaming Multiprocessor (SMs)**, each consisting of multiple **cores** with shared **control** and **memory**

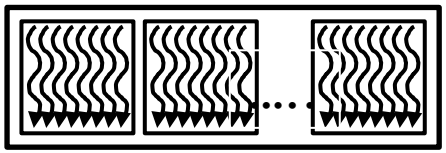
(e.g., a Volta V100 GPU has 80 SMs with 64 cores each which totals 5120 cores)



Assigning Blocks to SMs

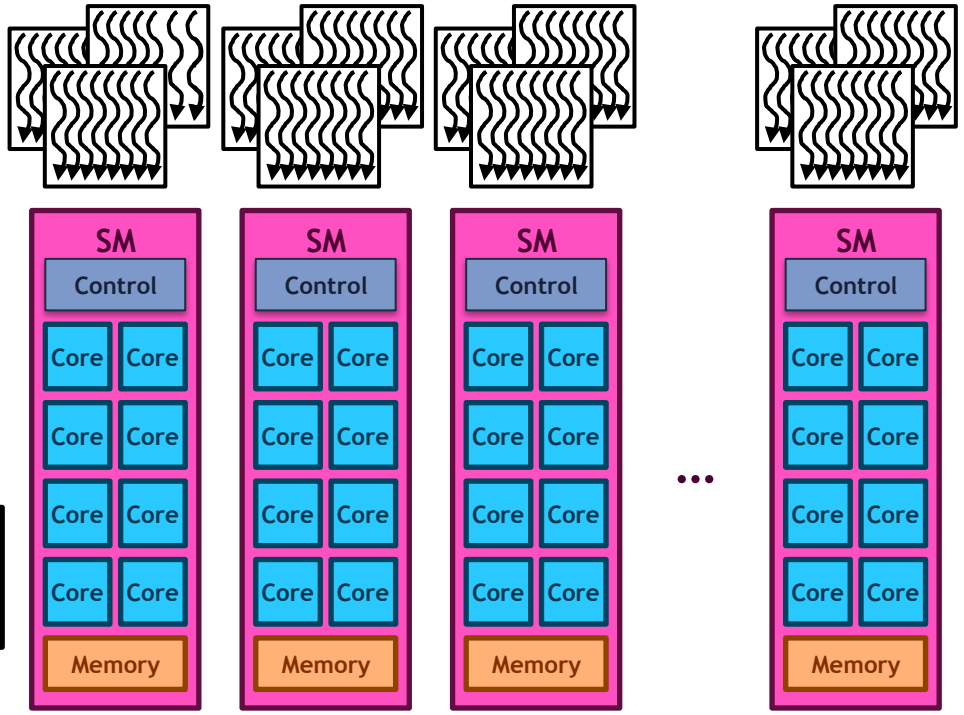
Threads/blocks require resources to execute (e.g., registers, memory, …) so SMs can accommodate a limited number of threads/ blocks at once…

… The remaining blocks wait for other blocks to finish before they can be assigned to an SM.



Waiting

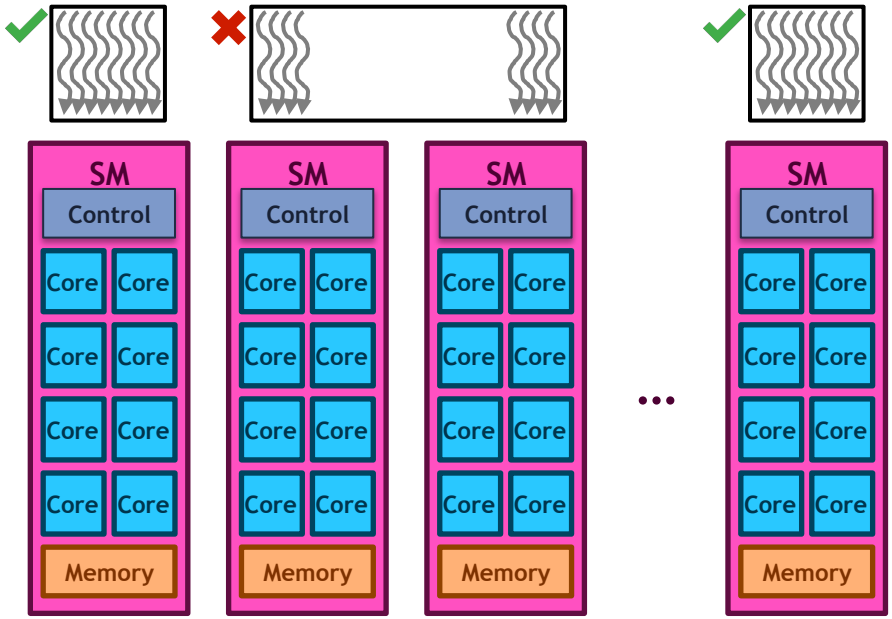
Threads are assigned to SMs at block granularity (all threads in a block are assigned to the same SM)



NVIDIA ILLINOIS

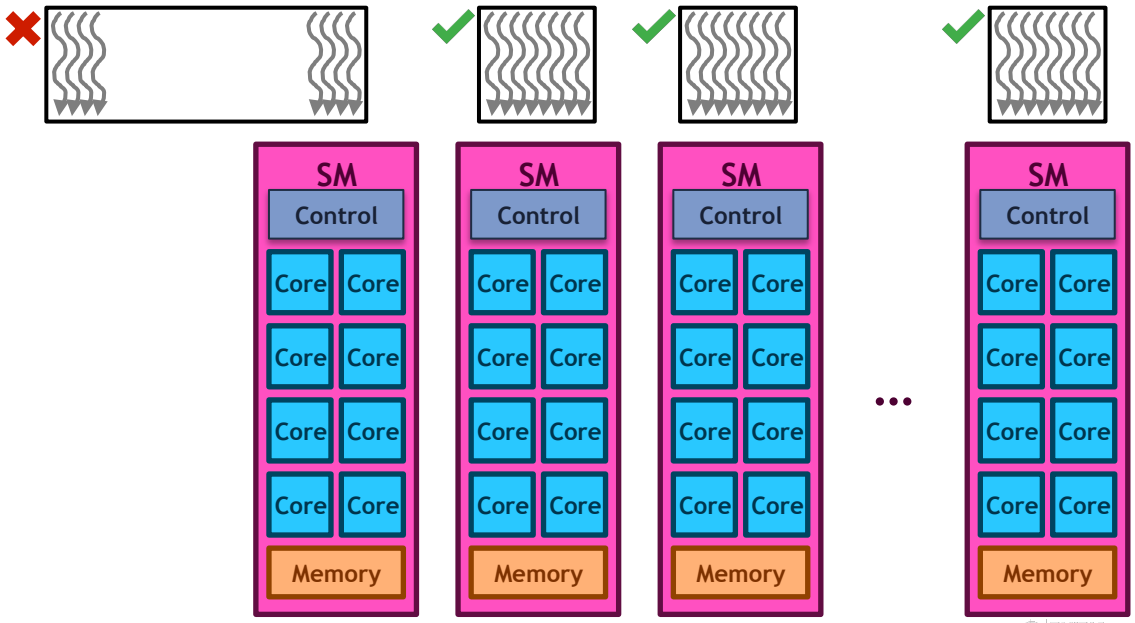
Scheduling Considerations

- Threads in the same block are assigned to the same SM
  - Assigning threads in the same block to the same SM makes supporting collaboration between them efficient



Scheduling Considerations

- All threads in a block are assigned to an SM simultaneously
  - A block cannot be assigned to an SM until it secures enough resources for all its threads to execute

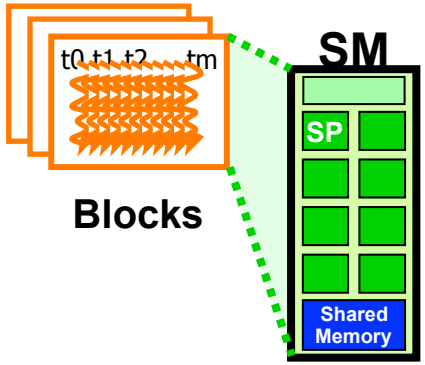


Synchronization

- Threads in the same block can collaborate in ways that threads in different blocks cannot:
  - Barrier synchronization: **__syncthreads()**
    - Wait for all threads in the block to reach the barrier before any thread can proceed
  - Shared memory (discussed later)
    - Access a fast memory that only threads in the same block can access
  - Others (discussed later)

Example: Executing Thread Blocks

- Threads are assigned to **Streaming Multiprocessors (SM)** in block granularity
  - Up to **8** blocks to each SM as resource allows
  - Fermi SM can take up to **1536** threads
    - Could be 256 (threads/block) * 6 blocks
    - Or 512 (threads/block) * 3 blocks, etc.



- SM maintains thread/block idx numbers
- SM manages/schedules thread execution

Block Granularity Considerations

- For Matrix Multiplication using multiple blocks, should I use 8X8, 16X16 or 32X32 blocks for Fermi?
  - For 8X8, we have 64 threads per Block. Since each SM can take up to 1536 threads, which translates to 24 Blocks. However, each SM can only take up to 8 Blocks, only 512 threads will go into each SM!
  - For 16X16, we have 256 threads per Block. Since each SM can take up to 1536 threads, it can take up to 6 Blocks and achieve full capacity unless other resource considerations overrule.
  - For 32X32, we would have 1024 threads per Block. Only one block can fit into an SM for Fermi. Using only 2/3 of the thread capacity of an SM.

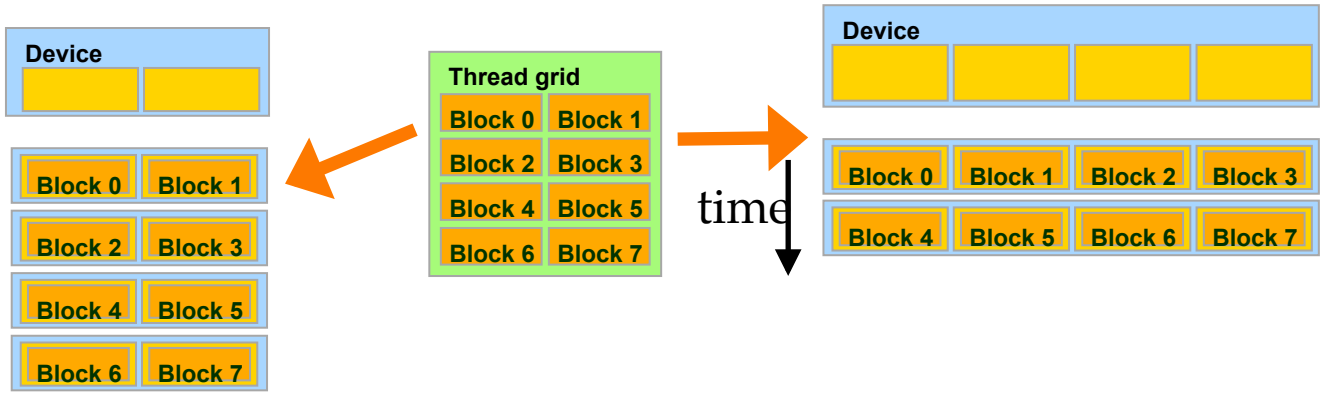
Transparent Scalability

- Threads in different blocks do not synchronize
  - Allows blocks to execute in any order
  - Allows blocks to execute both in parallel with each other or sequentially with respect to each other

- Enables **transparent scalability**
  - Same code can run on different devices with different amounts of hardware parallelism
    - Execute blocks sequentially if device has few SMs
    - Execute blocks in parallel if device has many SMs

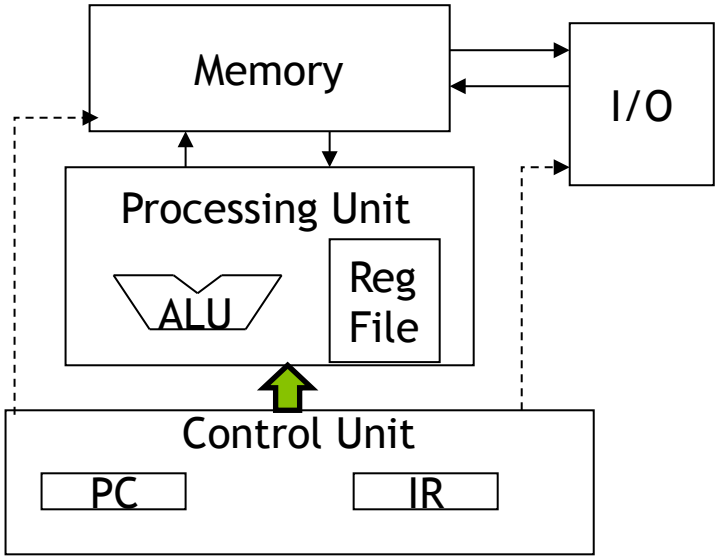
- Programmers should not write code that tries to synchronizes across blocks
  - Deadlock occurs if the blocks are not scheduled simultaneously
  - Note: ***cooperative groups*** is an advanced feature of CUDA that allows synchronization across blocks by limiting the number of blocks to guarantee that all blocks are executing simultaneously

Transparent Scalability

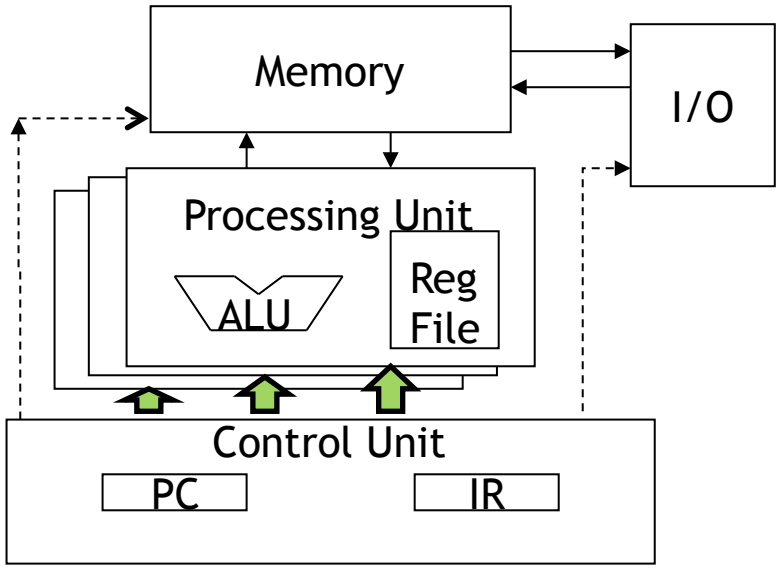


- Each block can execute in any order relative to others
- Hardware is free to assign blocks to any processor at any time
  - A kernel scales to any number of parallel processors

The Von-Neumann Model



The Von-Neumann Model with SIMD units

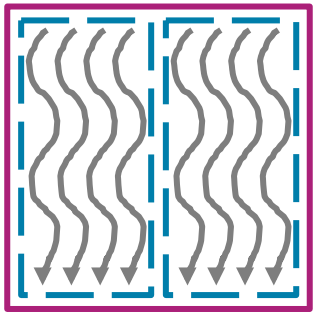


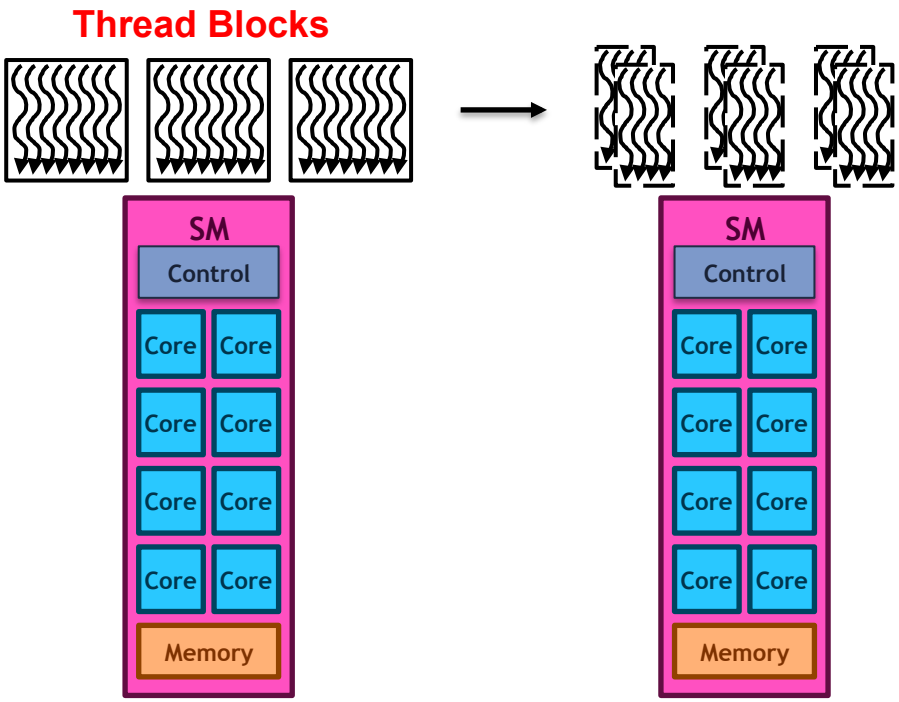
Single Instruction Multiple Data
(SIMD)

Thread Scheduling

- Threads assigned to an SM run concurrently
  - The SM has a scheduler that manages their execution

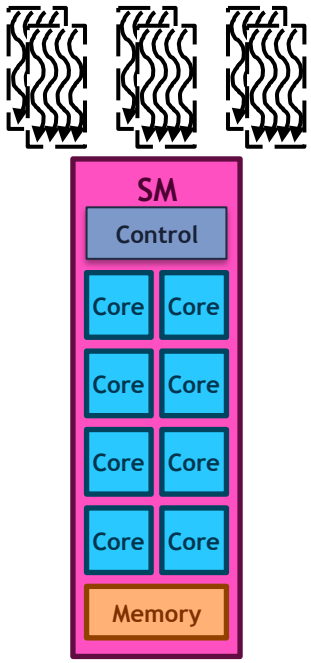
Blocks assigned to an SM are further divided into **warps** which are the unit of scheduling





Warps

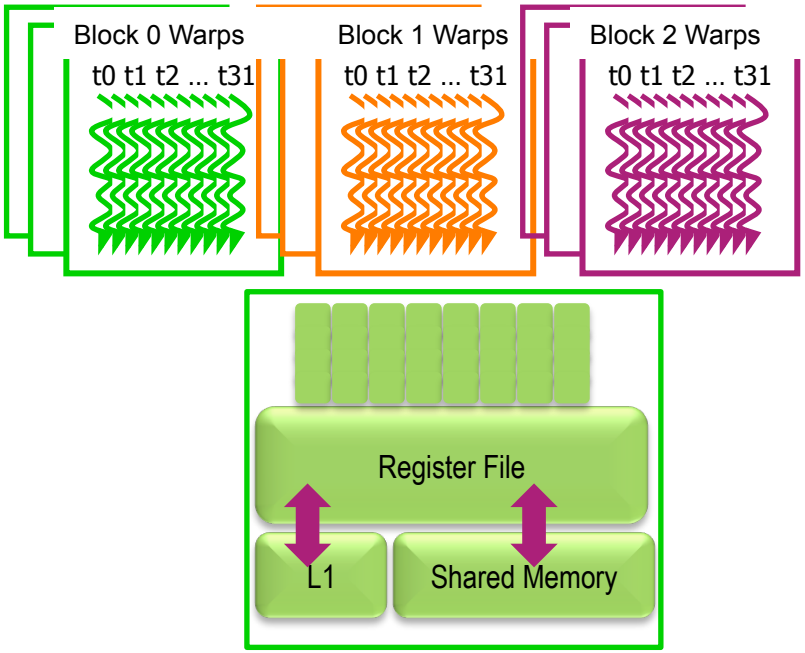
- Warps are the unit of scheduling in an SM
  - The size of warps is device-specific, but has always been 32 threads to date
  - An implementation decision, not part of the CUDA programming model
  - Threads in a warp execute in SIMD
  - Future GPUs may have different number of threads in each warp



- Threads in a warp are scheduled together and executed following the **SIMD** model
  - Single Instruction, Multiple Data
  - One instruction is fetched and executed by all the threads in the warp, each processing different data

Warp Example

- If 3 blocks are assigned to an SM and each block has 256 threads, how many Warps are there in an SM?
  - Each Block is divided into 256/32 = 8 Warps
  - There are 8 * 3 = 24 Warps



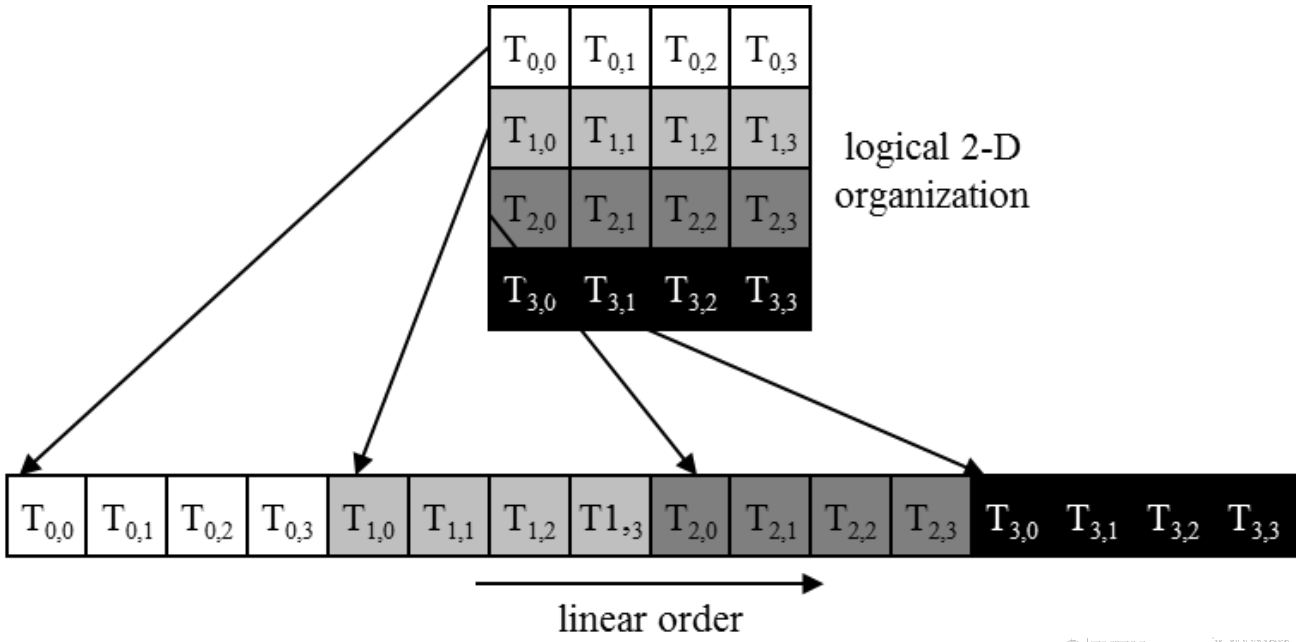
Example: Thread Scheduling (Cont.)

- SM implements zero-overhead warp scheduling
  - Warps whose next instruction has its operands ready for consumption are eligible for execution
  - Eligible Warps are selected for execution based on a prioritized scheduling policy
  - All threads in a warp execute the same instruction when selected

Warps in Multi-dimensional Thread Blocks

- The thread blocks are first linearized into 1D in row major order
  - In x-dimension first, y-dimension next, and z-dimension last

Placing 2D threads into linear order

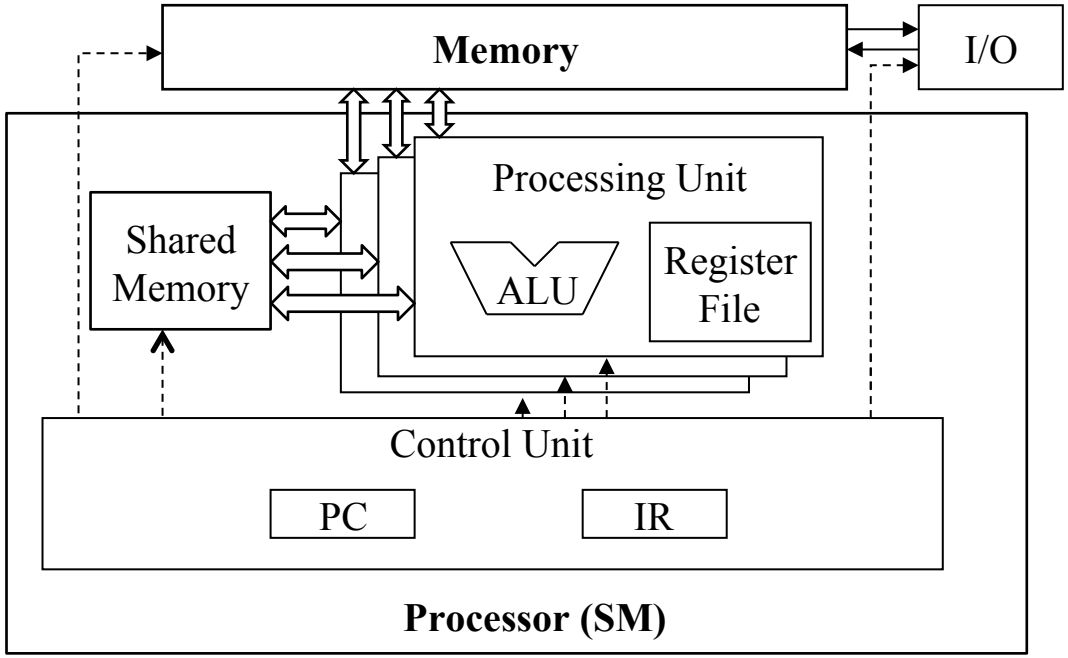


Blocks are partitioned after linearization

- Linearized thread blocks are partitioned
  - Thread indices within a warp are consecutive and increasing
  - Warp 0 starts with Thread 0
- Partitioning scheme is consistent across devices
  - Thus you can use this knowledge in control flow
  - However, the exact size of warps may change from generation to generation
- DO NOT rely on any ordering within or between warps
  - If there are any dependencies between threads, you must use __syncthreads() to get correct results (more later)

SMs are SIMD Processors

- Control unit for instruction fetch, decode, and control is shared among multiple processing units
  - Control overhead is minimized (Module 1)



SIMD Execution Among Threads in a Warp

- All threads in a warp must execute the same instruction at any point in time
- This works efficiently if all threads follow the same control flow path
  - All if-then-else statements make the same decision
  - All loops iterate the same number of times

Control Divergence

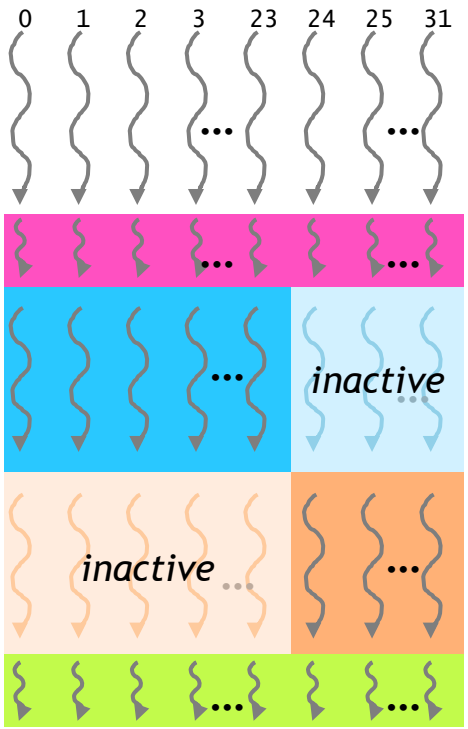
- Control divergence occurs when threads in a warp take different control flow paths by making different control decisions
  - Some take the then-path and others take the else-path of an if-statement
  - Some threads take different number of loop iterations than others

- The execution of threads taking different paths are serialized in current GPUs
  - The control paths taken by the threads in a warp are traversed one at a time until there is no more.
  - During the execution of each path, all threads taking that path will be executed in parallel
  - The number of different paths can be large when considering nested control flow statements

Example: Control Divergence

Example 1: Threads take different branches of a conditional

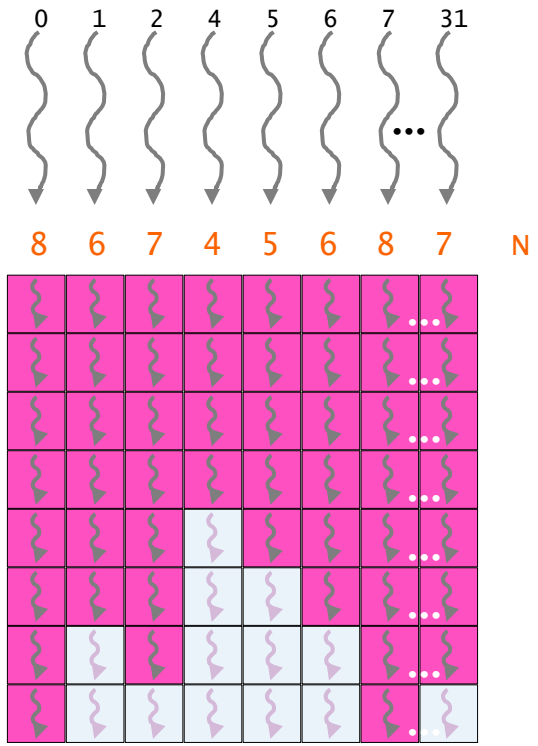
```
if(threadIdx.x < 24) {
    A
} else {
    B
}
C
```



Example: Control Divergence

Example 2: Threads execute a different number of loop iterations

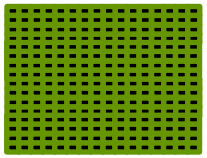
```
N = a[threadIdx.x];
for(i = 0; i < N; ++i) {
    A
}
```



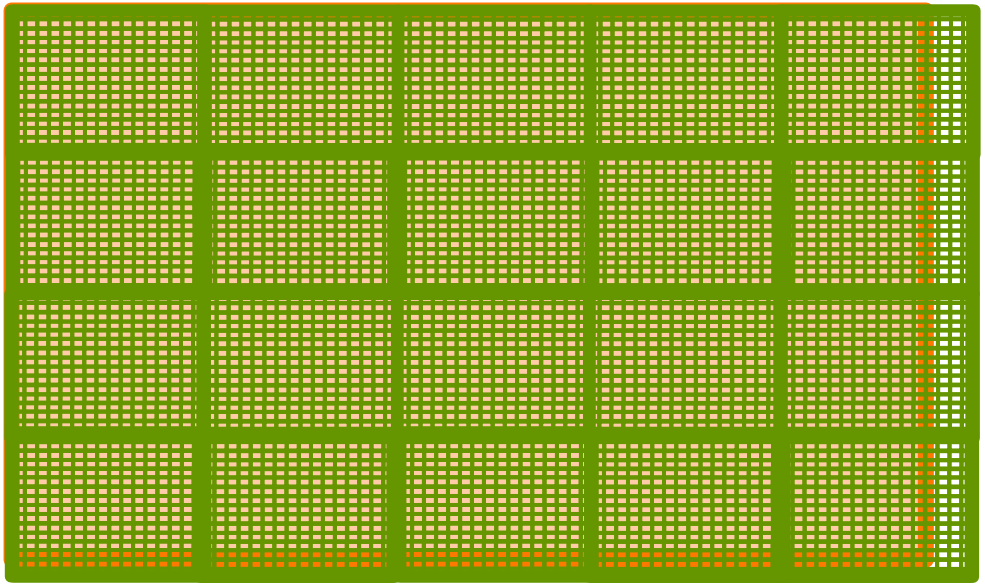
Control Divergence Examples

- Divergence can arise when branch or loop condition is a function of thread indices
- Example kernel statement with divergence:
  - if (threadIdx.x > 2) { }
  - This creates two different control paths for threads in a block
  - Decision granularity < warp size; threads 0, 1 and 2 follow different path than the rest of the threads in the first warp
- Example without divergence:
  - If (blockIdx.x > 2) { }
  - Decision granularity is a multiple of blocks size; all threads in any given warp follow the same path

An example for Control Divergence: Processing a Picture with a 2D Grid



16 16 blocks



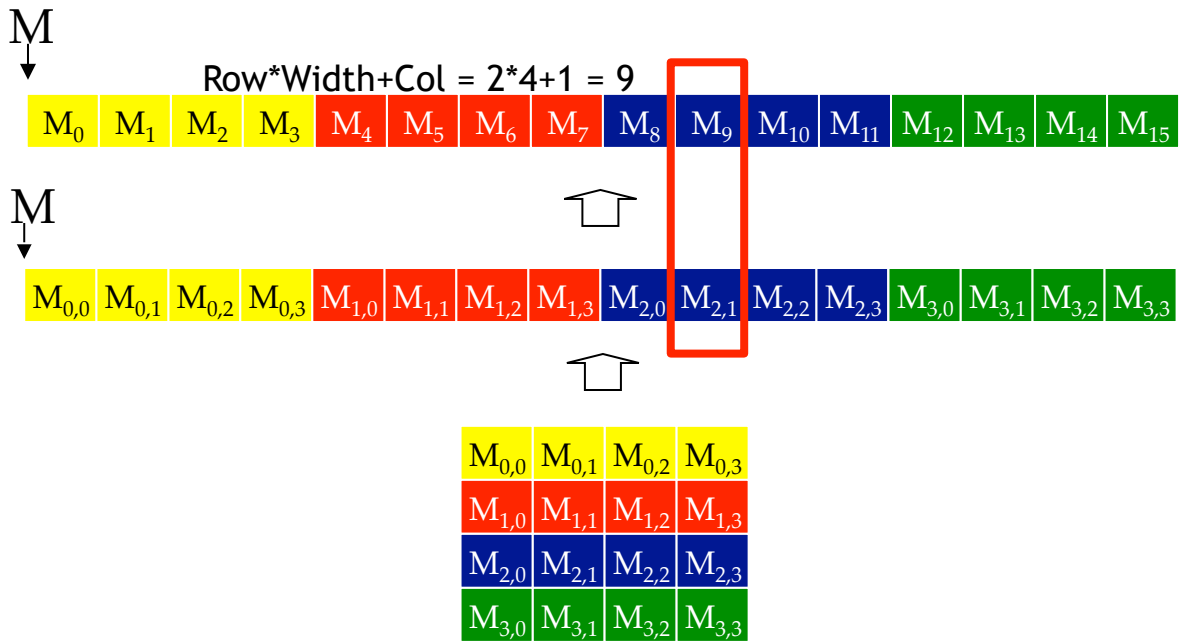
62 76 picture

Source Code of a PictureKernel

```
__global__ void PictureKernel(float* d_Pin, float* d_Pout,
                              int height, int width)
{

  // Calculate the row # of the d_Pin and d_Pout element
  int Row = blockIdx.y*blockDim.y + threadIdx.y;

  // Calculate the column # of the d_Pin and d_Pout element
  int Col = blockIdx.x*blockDim.x + threadIdx.x;

  // each thread computes one element of d_Pout if in range
  if ((Row < height) && (Col < width)) {
    d_Pout[Row*width+Col] = 2.0*d_Pin[Row*width+Col];
  }
}
```

Scale every pixel value by 2.0

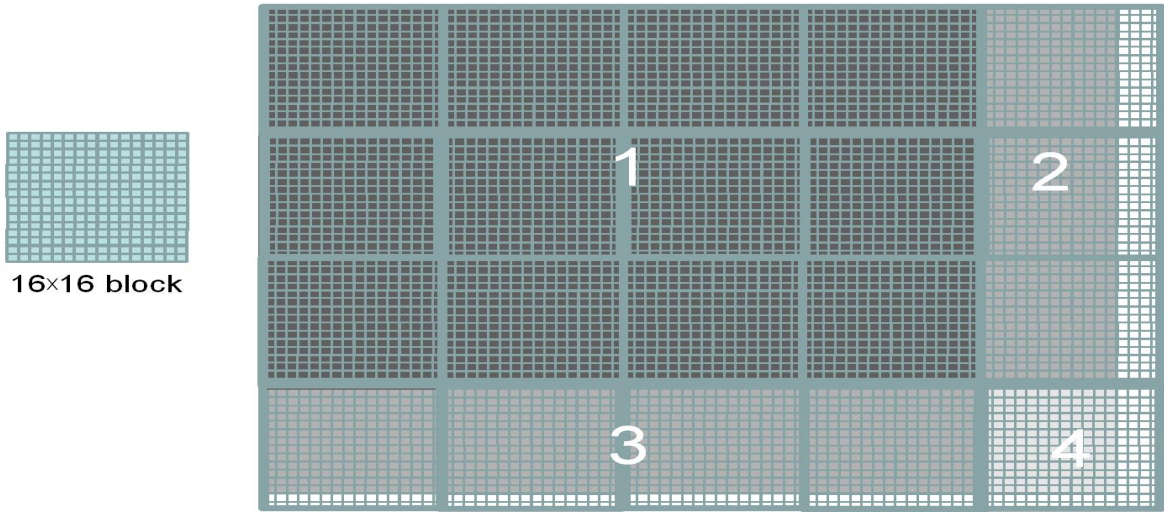
Row-Major Layout in C/C++



Host Code for Launching PictureKernel

```
// assume that the picture is mn,
// m pixels in y dimension and n pixels in x dimension
// input d_Pin has been allocated on and copied to device
// output d_Pout has been allocated on device
…
dim3 DimGrid((n-1)/16 + 1, (m-1)/16+1, 1);
dim3 DimBlock(16, 16, 1);
PictureKernel<<<DimGrid,DimBlock>>>(d_Pin, d_Pout, m, n);
…
```

Covering a 62x76 Picture with 16x16 Blocks



Not all threads in a Block will follow the same control flow path.

Another Example: Vector Addition Kernel

Device Code

```
// Compute vector sum C = A + B
// Each thread performs one pair-wise addition

__global__
void vecAddKernel(float* A, float* B, float* C,
  int n)
{
  int i = threadIdx.x + blockDim.x * blockIdx.x;
   if(i<n) C[i] = A[i] + B[i];
}
```

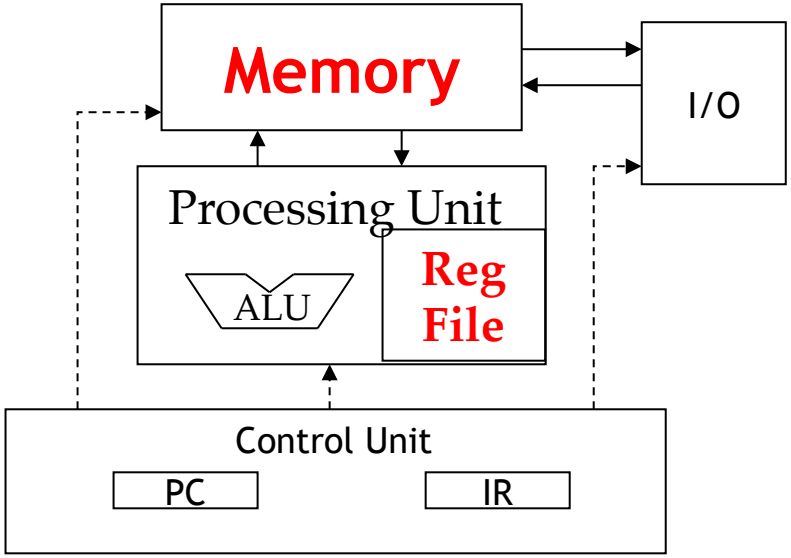
Analysis for vector size of 1,000 elements

- Assume that block size is 256 threads
  - 8 warps in each block
- All threads in Blocks 0, 1, and 2 are within valid range
  - i values from 0 to 767
  - There are 24 warps in these three blocks, none will have control divergence
- Most warps in Block 3 will not control divergence
  - Threads in the warps 0-6 are all within valid range, thus no control divergence

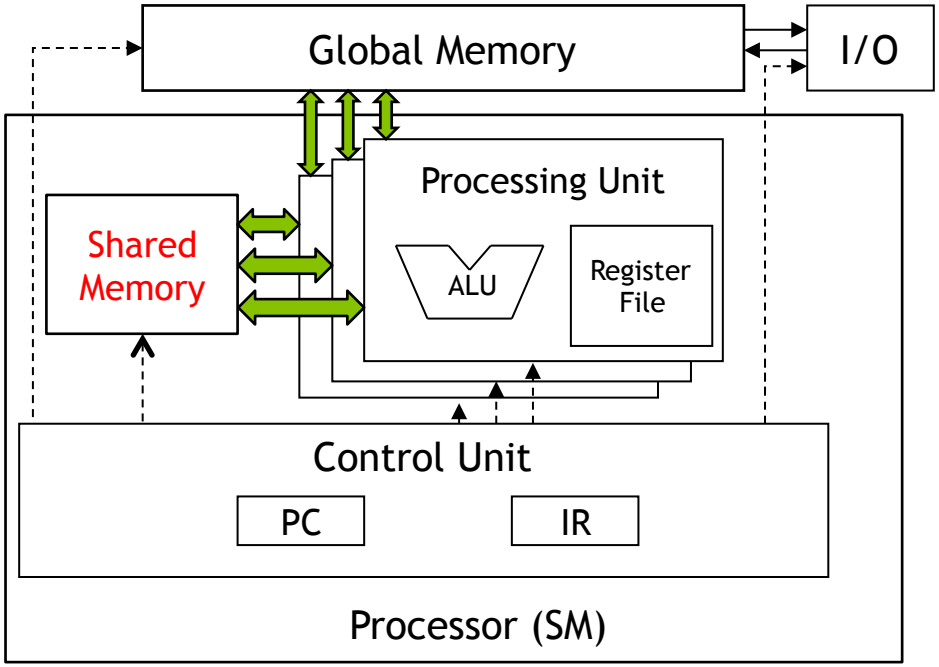
Analysis for vector size of 1,000 elements

- One warp in Block 3 will have control divergence
  - Threads with i values 992-999 will all be within valid range
  - Threads with i values of 1000-1023 will be outside valid range
- Effect of serialization on control divergence will be small
  - 1 out of 32 warps has control divergence
  - The impact on performance will likely be less than 3%

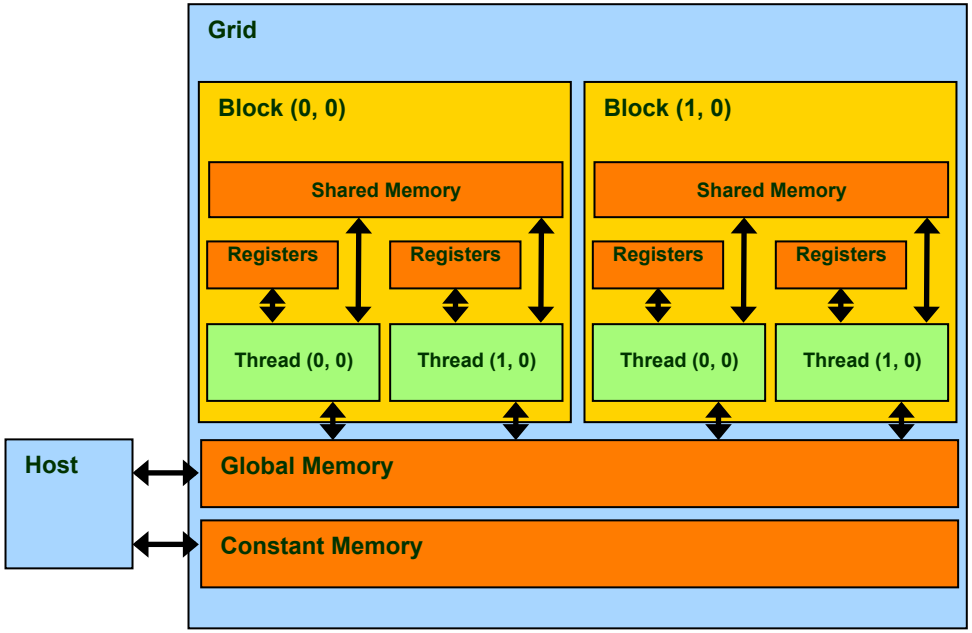
Memory and Registers in the Von-Neumann Model



Hardware View of CUDA Memories



Programmer View of CUDA Memories



Programmer View of CUDA Memories

- Global, constant, and texture memory contents are persistent across kernels called by the same application
- Each thread uses a number of registers **(thread private)**
- Each block allocates a certain amount of Shared Memory that is accessible to **all threads in the block (block private)**

Declaring CUDA Variables

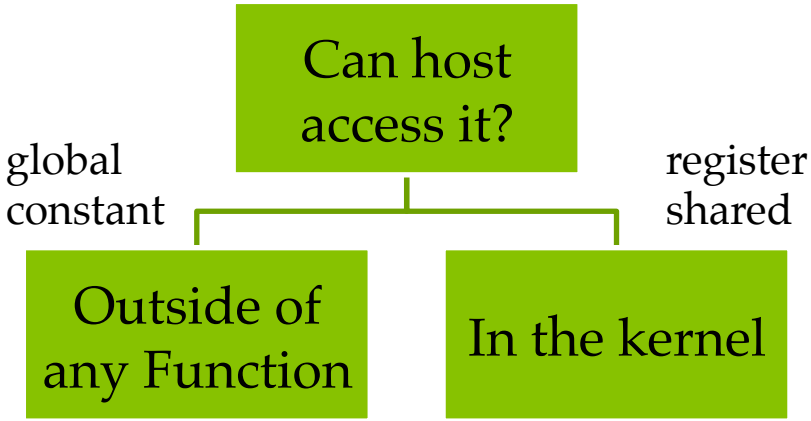
Variable declaration	Memory	Scope	Lifetime
int LocalVar;	register	thread	thread
__device__ __shared__ int SharedVar;	shared	block	block
__device__ int GlobalVar;	global	grid	application
__device__ __constant__ int ConstantVar;	constant	grid	application

- **`__device__`** is optional when used with **`__shared__`**, or **`__constant__`**
- Automatic variables reside in a register
  - Except per-thread arrays that reside in global memory

Example:
Shared Memory Variable Declaration

```
__global__
void blurKernel(unsigned char * in, unsigned char * out,
int w, int h)
{

  __shared__ float ds_in[TILE_WIDTH][TILE_WIDTH];

 …
}
```

Where to Declare Variables?



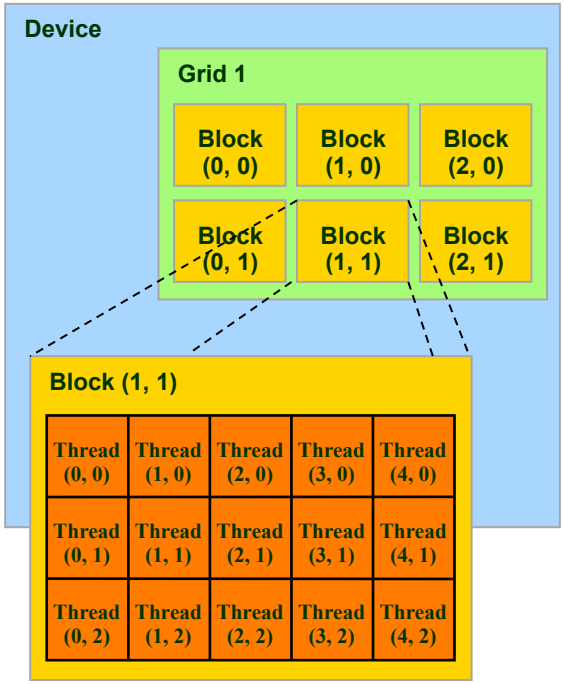
Shared Memory in CUDA

- A special type of memory whose contents are explicitly defined and used in the kernel source code
  - One in each SM
  - Accessed at much higher speed (in both latency and throughput) than global memory
  - Scope of access and sharing - thread blocks
  - Lifetime – thread block, contents will disappear after the corresponding thread finishes terminates execution
  - Accessed by memory load/store instructions
  - A form of scratchpad memory in computer architecture

CUDA Warps and Occupancy

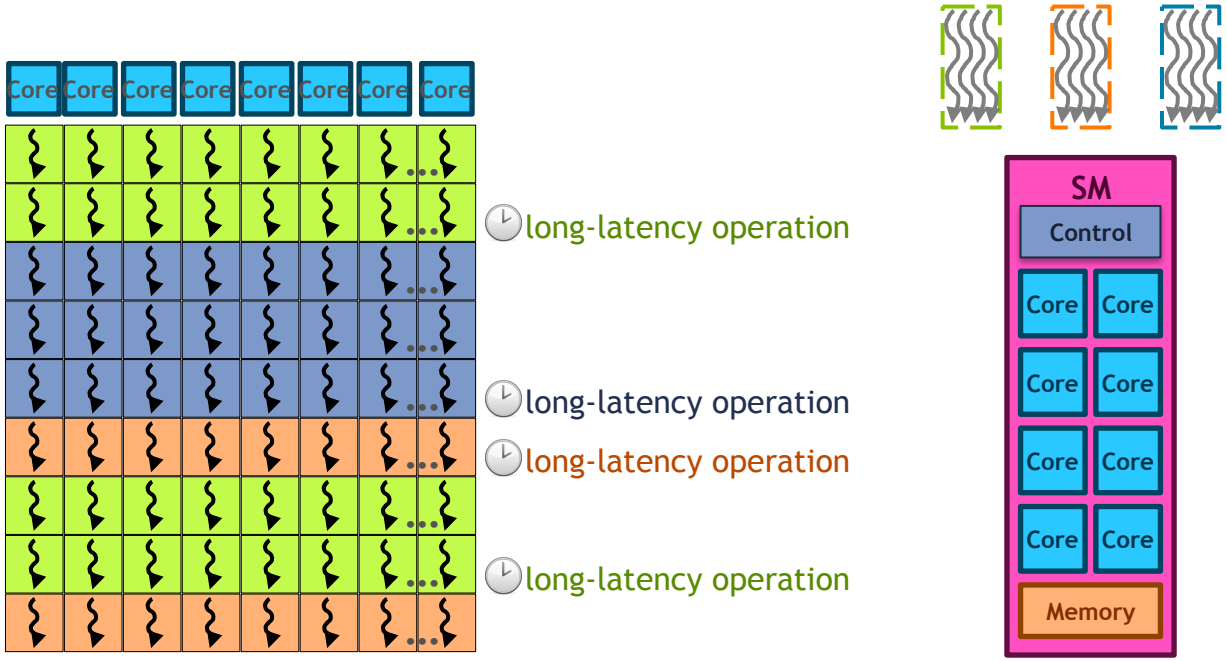
CUDA Programming Model Review

- A grid is composed of blocks which are completely independent
- A block is composed of threads which can communicate within their own block
- 32 threads form a warp
- Instructions are issued per warp
- If an operand is not ready the warp will stall
  - Context switch between warps when stalled
  - Context switch must be very fast



Latency Hiding

- When a warp needs to wait for a high latency operation, another warp that is ready is selected and scheduled for execution



Latency Hiding

- When a warp needs to wait for a high latency operation, another warp that is ready is selected and scheduled for execution
- Many warps are needed so that there is sufficient work available to hide long latency operations
  - i.e., there is high chance of finding a warp that is ready
- For this reason, an SM typically supports many more threads than the number of cores it has

SM Resources

GPU Name	K40	M40	P100	V100
Microarchitecture	Kepler	Maxwell	Pascal	Volta
Compute Capability	3.5	5.2	6.0	7.0
Number of SMs	15	24	56	80
Cores per SM	192	128	64	64
Max threads per SM	2048	2048	2048	2048
Max blocks per SM	16	32	32	32
Max threads per block	1024	1024	1024	1024
Registers per SM	64K	64K	64K	64K
Shared memory per SM	48KB	96KB	64KB	96KB

Max threads per SM is much higher than cores per SM

Fast Context Switching

- Registers and shared memory are allocated for a block as long as that block is active
  - Once a block is active it will stay active until all threads in that block have completed
  - Context switching is very fast because registers and shared memory do not need to be saved and restored
- **Goal: Have enough transactions in flight to saturate the memory bus**
  - Latency can be hidden by having more transactions in flight
  - Increase active threads or Instruction Level Parallelism (ILP)
- For example: Fermi can have up to 48 active warps per SM (1536 threads)

GPU Optimization Fundamentals

- Find ways to parallelize sequential code

- Kernel optimizations
  - Launch configuration
  - Global memory throughput
  - Shared memory access
  - Instruction throughput/control flow optimizations

- Optimization of CPU-GPU interaction
  - Maximizing PCIe throughput
  - Overlapping kernel execution with memory copies

Kernel Optimizations: Launch Configuration

- Launch configuration is the number of blocks and number of threads per block, expressed in CUDA with the <<< >>> notation:

```
mykernel<<<num_blocks,threads_per_block>>>(...);
```

- What values should we pick for these?
  - Need enough total threads to process entire input
  - Need enough threads to keep the GPU busy
  - Selection of block size is an optimization step involving warp occupancy

Occupancy

- Need enough independent threads per SM to hide latencies:
  - Instruction latencies
  - Memory access latencies
- **Occupancy = Active Warps / Maximum Active Warps**
- Remember: resources are allocated for the entire block
  - Resources are finite
  - Utilizing too many resources per thread may limit the occupancy

Occupancy

- Potential occupancy limiters:
  - Register usage
    - SM registers are partitioned among the threads
  - Shared memory usage
    - SM shared memory is partitioned among the blocks
  - Block size
    - Threads are allocated at thread block granularity

Occupancy Limiters: Registers

- Register usage: compile with --ptxas-options=-v
  - Reports shared memory and register usage
- Fermi has 32K registers per SM
- Example 1
  - Kernel uses 21 registers per thread
  - Active threads = 32K/21 = 1560 threads
    - > 1536 thus an occupancy of 1
- Example 2
  - Kernel uses 64 registers per thread
  - Active threads = 32K/64 = 512 threads
  - 512/1536 = 0.3333 occupancy
- Can control register usage with the nvcc flag: --maxrregcount

Occupancy Limiters: Shared Memory

- Shared memory usage: compile with --ptxas-options=-v
- Fermi has either 16K or 48K shared memory
- Example 1: 48K shared memory
  - Kernel uses 32 bytes of shared memory per thread
  - 48K/32 = 1536 threads
  - occupancy=1
- Example 2: 16K shared memory
  - Kernel uses 32 bytes of shared memory per thread
  - 16K/32 = 512 threads
  - occupancy=0.3333
- Don't use too much shared memory
- Choose L1/Shared configuration appropriately

Occupancy Limiter: Block Size

- For example: Each SM can have up to 8 active blocks
- A small block size will limit the total number of threads

Block Size	Active Threads	Occupancy
32	256	.1666
64	512	.3333
128	1024	.6666
192	1536	1
256	2048 (1536)	1

- Avoid small block sizes, generally 128-256 threads is sufficient

SM Resources

GPU Name	K40	M40	P100	V100
Microarchitecture	Kepler	Maxwell	Pascal	Volta
Compute Capability	3.5	5.2	6.0	7.0
Number of SMs	15	24	56	80
Cores per SM	192	128	64	64
Max threads per SM	2048	2048	2048	2048
Max blocks per SM	16	32	32	32
Max threads per block	1024	1024	1024	1024
Registers per SM	64K	64K	64K	64K
Shared memory per SM	48KB	96KB	64KB	96KB

Max threads per SM is much higher than cores per SM

General Guidelines

- Thread block size choice:
  - Start with 128-256 threads per block
    - Adjust up/down by what best matches your function
    - Example: stencil codes prefer larger blocks to minimize halos
  - Multiple of warp size (32 threads)
  - If occupancy is critical to performance:
    - Check that block size isn't preventing from occupancy allowed by register and shared memory resources
- Grid size:
  - 1,000 or more thread blocks
    - 10s of "waves" of thread blocks
    - Makes your code ready for several generations of future GPUs

Occupancy and Performance

- Note that 100% occupancy isn't needed to reach maximum performance
  - Once the "needed" occupancy is reached, further increases won't improve performance

- Needed occupancy depends on the code
  - More independent work per thread -> less occupancy is needed
  - Memory-bound codes tend to need more occupancy
    - Higher latency than for arithmetic, need more work to hide it

What Occupancy Do I Need?

- Depends on your problem...
  - Many find 66% is enough to saturate the bandwidth
- **Look at increasing occupancy only if the following are true!**
  - The kernel is bandwidth bound
  - The achieved bandwidth is significantly less than peak
- Instruction Level Parallelism (ILP) can have a greater effect than increasing occupancy

Summary of the Section

- In order to achieve peak global memory bandwidth we need to have enough transactions in flight to hide latency

- We can increase the number of transactions by
  - Increasing occupancy
  - Increasing instruction level parallelism

- Occupancy can be limited by
  - Register usage
  - Shared memory usage
  - Block size

- Use the visual profiler to investigate memory bandwidth/occupancy